

M7 Math Note

Chung-En (Johnny) Yu

Department of Intelligent Systems and Robotics, University of West Florida

Tip

This math note serves as supplementary content for the lecture. Students are encouraged to read it in conjunction with the lecture.

Ensemble Learning

Ensemble learning is a machine learning paradigm where multiple models (often called "weak learners") are trained to solve the same problem and combined to get better results. The main premise is that by combining the predictions of several models, the ensemble can often outperform any single model in the ensemble.

Key principles

- **Diversity:** Effective ensembles combine models that make different kinds of errors, which can reduce the overall error of the ensemble.
- **Aggregation:** The ensemble makes predictions based on a weighted vote or average of its members' predictions.

Methods in ensemble learning

- Bagging (Bootstrap Aggregating)
- Boosting

Majority Vote

Using the training dataset, we start by training m different classifiers, $(C_1, \dots, C_m)$.

To predict a class label via simple majority voting:

1. Combine the predicted class labels of each individual classifier, C_j
2. Select the class label, $\hat{y}$, that received the most votes

$$\hat{y} = \text{mode} \{C_1(\mathbf{x}), C_2(\mathbf{x}), \dots, C_m(\mathbf{x})\}$$

- In statistics, the **mode** is the most frequent event or result in a set. For example, $\text{mode}\{1, 2, 1, 1, 2, 4, 5, 4\} = 1$.

For example, in a binary classification task where class 1 = -1 and class 2 = +1, we can write the majority vote prediction as follows:

$$C(\mathbf{x}) = \text{sign} \left[\sum_j^m C_j(\mathbf{x}) \right] = \begin{cases} 1, & \text{if } \sum_j C_j(\mathbf{x}) \geq 0 \\ -1, & \text{otherwise} \end{cases}$$

Ensemble error

To prove why ensemble better than single classifier.

Assumptions

- All n -base classifiers for a binary classification task have an equal *error rate*, ε .
- The classifiers are independent, and the error rates are not correlated.

Error probability of an ensemble of base classifiers

Describe it as a probability mass function of a binomial distribution:

$$P(y \geq k) = \sum_k^n \binom{n}{k} \varepsilon^k (1 - \varepsilon)^{n-k} = \varepsilon_{\text{ensemble}}$$

- $\binom{n}{k} = \frac{n!}{(n-k)!k!}$: binomial coefficient, n choose k
 - We compute the probability that the prediction of the ensemble is wrong.

Example

Say there are 11 base classifiers ($n = 11$), where each classifier has an error rate of 0.25 ($\varepsilon = 0.25$) :

$$P(y \geq k) = \sum_{k=6}^{11} \binom{11}{k} 0.25^k (1 - 0.25)^{11-k} = 0.034$$

As you can see, the error rate of the ensemble (0.034) is much lower than the error rate of each individual classifier (0.25) if all the assumptions are met. Note that, in this simplified illustration, a 50-50 split by an even number of classifiers, n , is treated as an error, whereas this is only true half of the time.

Weighted Majority Vote Classifier

To combine different classification algorithms associated with individual weights for confidence. Build a stronger meta-classifier that balances out the individual classifiers' weaknesses on a particular dataset.

The general weighted majority vote can be written as follows:

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j \chi_A (C_j(\mathbf{x}) = i)$$

- w_j : Weight associated with a base classifier, C_j .
- $\hat{y}$: The predicted class label of the ensemble.
- A : The set of unique class labels.

- χ_A : The characteristic function or indicator function.
 - returns 1 if the predicted class of the j -th classifier matches i ($C_j(x) = i$)

For equal weights, simplify this equation:

$$\hat{y} = \text{mode} \{C_1(\mathbf{x}), C_2(\mathbf{x}), \dots, C_m(\mathbf{x})\}$$

Using the **predicted class probabilities** instead of the class labels for majority voting can be useful if the classifiers in our ensemble are well calibrated:

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j p_{ij}$$

- p_{ij} : predicted probability of the j -th classifier for class label i

Bagging (Bootstrap Aggregating)

It involves training multiple models in parallel, each on a random subset of the data. Instead of using the same training dataset to fit the individual classifiers in the ensemble, we draw **bootstrap samples (= random samples with replacement)** from the initial training dataset.

- **Pros:**
 - Reduces variance and helps to avoid overfitting, making the model more robust.
 - Can be easily parallelized, as models are built independently.
 - More robust than boosting when data is noisy.
- **Cons:**
 - Models can become very similar to each other if the underlying training data is not diverse enough.
 - Ineffective in reducing model bias (models that are too simple to capture the trends in the data well).
 - Increased computational cost due to training multiple models.

Boosting

Sequential training of models where each model tries to correct the errors of its predecessors.

- **Pros:**
 - Often provides significant accuracy improvements over single models.
 - Can handle different types of data and incorporate various underlying learners.
 - Focuses on reducing bias as well as variance, which can lead to better performance on imbalanced datasets.
- **Cons:**
 - More prone to overfitting compared to bagging if the data is noisy.

- Sequential nature of boosting can make it harder to parallelize and thus can be slower to train than bagging methods.
- Requires careful tuning of parameters and stopping criteria to prevent overfitting.

Adaptive Boosting (AdaBoost)

AdaBoost uses the complete training dataset to train the weak learners, where the training examples are reweighted in each iteration to build a strong classifier that learns from the mistakes of the previous weak learners in the ensemble.

AdaBoost Algorithm

1. Set the weight vector, w , to uniform weights, where $\sum_i w_i = 1$.
2. For j in m boosting rounds, do the following:
 - a. Train a weighted weak learner: $C_j = \text{train}(X, y, w)$.
 - b. Predict class labels: $\hat{y} = \text{predict}(C_j, X)$.
 - c. Compute the weighted error rate: $\varepsilon = w \cdot (\hat{y} \neq y)$.
 - d. Compute the coefficient: $\alpha_j = 0.5 \log \frac{1-\varepsilon}{\varepsilon}$.
 - e. Update the weights: $w := w \times \exp(-\alpha_j \times \hat{y} \times y)$.
 - f. Normalize the weights to sum to 1: $w := w / \sum_i w_i$.
3. Compute the final prediction: $\hat{y} = \left(\sum_{j=1}^m (\alpha_j \times \text{predict}(C_j, X)) > 0 \right)$.

Note that

- element-wise multiplication: cross symbol ($\times$)
- dot-product: dot symbol ($\cdot$)
- $(\hat{y} \neq y)$ in step 2c refers to a binary vector consisting of 1 s and 0 s,
 - 1 is assigned, if the prediction is incorrect
 - 0 is assigned, otherwise

AdaBoost vs gradient boosting

- **AdaBoost** trains decision tree stumps based on errors of the previous decision tree stump.
 - The errors are used to compute sample weights in each round as well as for computing a classifier weight for each decision tree stump when combining the individual stumps into an ensemble.
 - Stop training once a maximum number of iterations (decision tree stumps) is reached.
- **Gradient boosting** fits decision trees in an iterative fashion using prediction errors.
 - Gradient boosting trees are usually deeper than decision tree stumps and have typically a maximum depth of 3 to 6 (or a maximum number of 8 to 64 leaf nodes).
 - The prediction errors are used directly to form the target variable for fitting the next tree.
 - It uses a global learning rate that is the same for each tree.

Gradient Boosting

Gradient boosting builds a series of trees, where each tree is fit on the **error** (the difference between the label and the predicted value) of the previous tree. In each round, the tree ensemble improves as we are nudging each tree more in the right direction via small updates. These updates are based on a loss gradient, which is how gradient boosting got its name.

🔗 Gradient Boosting Algorithm

1. Initialize a model to return a **constant model prediction value**, $F_0(x)$.

$$F_0(x) = \arg \min_{\hat{y}} \sum_{i=1}^n L(y_i, \hat{y})$$

- We use a **decision tree root node** (= a decision tree with a single leaf node) for $F_0(x)$.
- y : class label
- $\hat{y}$: predicted value returned by the tree (model) by minimizing a differentiable loss function L .
- n : the n training examples

2. For each tree $m = 1, \dots, M$, where M is a user-specified total number of trees:

1. Compute the **pseudo-residual**, r_{im} .

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad \text{for } i = 1, \dots, n$$

- r_{im} : pseudo-residual = pseudo-response
- m : number of the tree
- $F(x)$: model prediction value, which is the prediction of the previous tree model, $F_{m-1}(x)$.
 - In the first round, this refers to $F_0(x)$ from the tree (single leaf node) from step 1.

2. Fit a tree to the pseudo-residuals, r_{im} .

3. For each leaf node, R_{jm} , we compute the following output value, γ_{jm} .

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$$

- R_{jm} : leaf node, where $j = 1 \dots J_m$ leaf nodes of the resulting tree in iteration m

4. Update the tree model by adding the output value, γ_m , to the previous tree.

$$F_m(x) = F_{m-1}(x) + \eta \gamma_m$$

- η : learning rate
 - update model by taking small steps $\rightarrow$ avoid overfitting
 - typically a small value between 0.01 and 1

Stacking (Stacked Generalization)

Stacking involves training multiple different models (could be of different types) and then using a meta-model to learn how to best combine their predictions. The **base level models** are trained on the training data. The **meta-model** is trained on the outputs of the base models as features.

- **Pros:**

- Can leverage the strengths of each individual model by combining their predictions.
- Often provides better predictive performance on large and complex datasets than any single model or other simpler ensemble methods.

- **Cons:**

- Model complexity can lead to difficulty in interpretation and understanding.
 - Risk of overfitting if the meta-model is too complex or if there is not enough separation between training of base models and the meta-model.
 - Can be computationally expensive and time-consuming to train, especially with large datasets and many base models.
-